# Development of a Real-Time IoE Stream Analytics and Anomaly Detection System using Node-RED and MQTT

## Objective:

To develop a real-time IoE stream analytics system that monitors temperature, vibration and current readings, applies threshold-based rules to detect anomalies, and raises ALERT events, following the flow of an MQTT and Node-RED style pipeline.

## Procedure:

1. Generate a simulated telemetry stream of 600 samples at 5-second intervals starting 21 Oct 2026, 10:00: temperature (mean 27 C, SD 2.0), vibration (mean 3.0 mm/s, SD 0.45) and current (mean 5.0 A, SD 0.7).
2. Inject 12 artificial faults: temperature +12 C at four samples, vibration +3.5 mm/s at four samples and current +4.0 A at four samples.
3. Apply threshold rules to every sample (the rule stage of a Node-RED flow): temperature above 35 C, vibration above 4.5 mm/s, or current above 7.0 A. A sample is marked ALERT if any rule is exceeded and NORMAL otherwise.
4. Count normal and anomalous samples, compute the anomaly rate and list the detected events.
5. Plot the temperature, vibration and current streams with their threshold lines and a normal-vs-anomaly summary, and export the results to IoE_Stream_Results.csv.

## Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

np.random.seed(23)

# 1. Generate simulated IoE telemetry stream
N = 600
start = datetime(2026, 10, 21, 10, 0)

timestamps = [
    start + timedelta(seconds=5*i) for i in range(N)
]
```

```
temperature = np.random.normal(27, 2.0, N)
vibration = np.random.normal(3.0, 0.45, N)
current = np.random.normal(5.0, 0.7, N)

# 2. Inject artificial anomalies
temperature[[75, 76, 77, 180]] += 12
vibration[[181, 182, 310, 311]] += 3.5
current[[450, 451, 452, 530]] += 4.0

df = pd.DataFrame({
    "timestamp": timestamps,
    "temperature_C": temperature.round(2),
    "vibration_mm_s": vibration.round(2),
    "current_A": current.round(2)
})

# 3. Threshold-based anomaly detection
df["temp_anomaly"] = df["temperature_C"] > 35
df["vibration_anomaly"] = df["vibration_mm_s"] > 4.5
df["current_anomaly"] = df["current_A"] > 7.0

df["anomaly"] = df[
    ["temp_anomaly", "vibration_anomaly", "current_anomaly"]
].any(axis=1)

df["status"] = np.where(df["anomaly"], "ALERT", "NORMAL")

anomalies = int(df["anomaly"].sum())
normal = N - anomalies

print("=" * 55)
print("IoE STREAM ANALYTICS AND ANOMALY DETECTION")
print("=" * 55)
print("Total samples:", N)
print("Normal samples:", normal)
print("Anomalous samples:", anomalies)
print(f"Anomaly rate: {anomalies/N*100:.2f}%")

print("\nThresholds:")
print("Temperature > 35 °C")
print("Vibration > 4.5 mm/s")
print("Current > 7.0 A")

print("\nSample detected events:")
print(df[df["anomaly"]][
    ["timestamp", "temperature_C", "vibration_mm_s",
     "current_A", "status"]
].head(10).to_string(index=False))

# 4. Plot stream analytics dashboard
fig, ax = plt.subplots(2, 2, figsize=(14, 9))

ax[0, 0].plot(df["timestamp"], df["temperature_C"])
ax[0, 0].axhline(35, linestyle="--", label="Threshold")
ax[0, 0].set_title("Temperature Stream")
ax[0, 0].set_ylabel("Temperature (°C)")
ax[0, 0].tick_params(axis="x", rotation=25)
ax[0, 0].legend()
ax[0, 0].grid(True, alpha=0.3)
```

```
ax[0, 1].plot(df["timestamp"], df["vibration_mm_s"])
ax[0, 1].axhline(4.5, linestyle="--", label="Threshold")
ax[0, 1].set_title("Vibration Stream")
ax[0, 1].set_ylabel("Vibration (mm/s)")
ax[0, 1].tick_params(axis="x", rotation=25)
ax[0, 1].legend()
ax[0, 1].grid(True, alpha=0.3)

ax[1, 0].plot(df["timestamp"], df["current_A"])
ax[1, 0].axhline(7.0, linestyle="--", label="Threshold")
ax[1, 0].set_title("Current Stream")
ax[1, 0].set_ylabel("Current (A)")
ax[1, 0].tick_params(axis="x", rotation=25)
ax[1, 0].legend()
ax[1, 0].grid(True, alpha=0.3)

ax[1, 1].bar(
    ["Normal", "Anomaly"],
    [normal, anomalies]
)
ax[1, 1].set_title("Anomaly Detection Summary")
ax[1, 1].set_ylabel("Number of Samples")
ax[1, 1].grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.savefig("IoE_Anomaly_Detection.png", dpi=180)
plt.show()

# 5. Export results for further analysis
df.to_csv("IoE_Stream_Results.csv", index=False)

print("\nSimulation completed successfully.")
print("Dashboard image: IoE_Anomaly_Detection.png")
print("Dataset: IoE_Stream_Results.csv")

# Download files when running in Google Colab
from google.colab import files
files.download("IoE_Anomaly_Detection.png")
files.download("IoE_Stream_Results.csv")
```

## Output:

```
======================================================
IoE STREAM ANALYTICS AND ANOMALY DETECTION
======================================================
Total samples: 600
Normal samples: 587
Anomalous samples: 13
Anomaly rate: 2.17%

Thresholds:
Temperature > 35 °C
Vibration > 4.5 mm/s
Current > 7.0 A

Sample detected events:
            timestamp  temperature_C  vibration_mm_s  current_A status
2026-10-21 10:06:15          37.58            2.31       5.44  ALERT
2026-10-21 10:06:20          38.73            2.92       4.04  ALERT
2026-10-21 10:06:25          38.97            3.53       5.13  ALERT
2026-10-21 10:15:00          40.73            3.31       5.23  ALERT
2026-10-21 10:15:05          27.70            6.12       4.65  ALERT
2026-10-21 10:15:10          24.50            6.78       5.15  ALERT
2026-10-21 10:16:35          27.38            2.49       7.29  ALERT
2026-10-21 10:25:50          29.01            6.00       5.89  ALERT
2026-10-21 10:25:55          28.60            6.22       5.31  ALERT
2026-10-21 10:37:30          29.00            2.84       8.73  ALERT
```

*Fig. 1: Console output (summary, thresholds and detected events)*

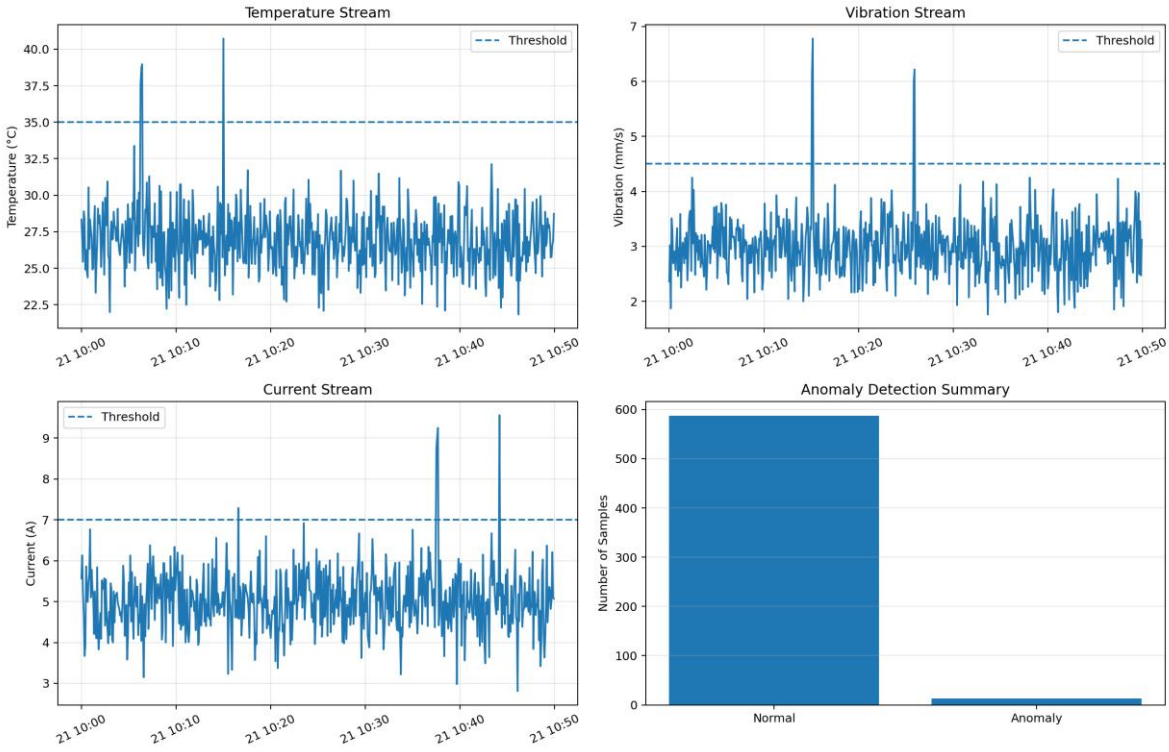



*Fig. 2: Stream analytics dashboard (temperature, vibration, current and anomaly summary)*

## Results:

| Metric | Value |
|---|---|
| Total samples | 600 |
| Normal samples | 587 |
| Anomalous samples (ALERT) | 13 |
| Anomaly rate | 2.17% |

| Signal | Threshold | Samples above threshold | Peak value |
|---|---|---|---|
| Temperature | > 35 °C | 4 | 40.73 °C |
| Vibration | > 4.5 mm/s | 4 | 6.78 mm/s |
| Current | > 7.0 A | 5 | 9.56 A |

13 of the 600 samples (2.17%) were flagged. All 12 injected faults were detected: 4 temperature, 4 vibration and 4 current spikes, plus one extra current alert at 10:16:35 (7.29 A) that came from normal random variation just above the 7.0 A threshold, which is a false alarm of a fixed-threshold rule. The alerts are clustered at 10:06, 10:15, 10:25, 10:37 and 10:44, and each is visible as a spike above the dashed threshold line in the graphs.

## Result:

The system classified the stream into 587 normal and 13 anomalous samples (2.17% anomaly rate) and flagged every injected fault in real time using simple threshold rules on temperature, vibration and current. Threshold detection is fast and easy to run in a Node-RED flow, although a fixed limit can raise an occasional false alarm, as the single extra current alert shows. All data are simulated, not measured from real equipment.